

ORIGINAL RESEARCH OPEN ACCESS

Specialized Deep Residual Policy Reinforcement Learning Framework for Safe and Adaptive Continuous Control

Ammar N. Abbas¹  | Georgios C. Chasparis² | John D. Kelleher³
¹Computer Science Department, Technological University Dublin, Dublin, Ireland | ²Data Science Division at the Software Competence Center Hagenberg, Hagenberg, Austria | ³ADAPT Research Centre, School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland

Correspondence: Ammar N. Abbas (ammar.abbas@tudublin.ie)

Received: 26 July 2025 | **Revised:** 13 November 2025 | **Accepted:** 26 December 2025

ABSTRACT

Traditional controllers have limitations as they rely on prior knowledge about the physics of the problem, require modelling of dynamics, and struggle to adapt to abnormal situations. Deep reinforcement learning (DRL) offers a promising alternative by learning policies through exploration, but its black-box nature and reliance on random exploration pose challenges in safety-critical environments. Recognizing that conventional controllers and DRL have complementary strengths, we propose a novel hybrid framework to overcome challenges in both conventional control systems and DRL. This framework integrates residual policy learning, a cycle of learning approach, and a specialized reinforcement learning agent for safety-critical, continuous control. Residual policy learning enables collaboration between DRL and conventional controllers, the cycle of learning improves learning efficiency by leveraging expert trajectories, and a specialized reinforcement learning agent optimizes policy learning in critical states using an input-output hidden Markov model. The framework is validated on the Tennessee Eastman process through experiments that analyse synchronization, activation mechanisms and an ablation study.

1 | Introduction

The principal objectives of a control system are to either: (i) track a setpoint as accurately as possible or (ii) respond to external disturbances and restore normal operation as efficiently as possible. The process industry predominantly relies on conventional controllers like model predictive control (MPC) and proportional-integral-derivative (PID) controllers. These controllers require precise physical or mathematical modelling. While effective, they are typically applied to isolated systems to overcome challenges associated with complex processes, such as unmodelled dynamics and multi-component interactions. Additionally, they require continuous maintenance and recalibration, as their performance can degrade over time due to system drift or variations in setpoints [1].

Deep reinforcement learning (DRL)—RL combined with deep neural networks (DNN)—has been successfully applied in prior

work to control continuous state-action spaces [2]. DRL can be classified as either model-free or model-based. In a model-free setting, the agent learns optimal policies directly from interactions with the environment, without relying on an explicit model of the environment's dynamics [3, 4]. DRL has been explored in process control to address the limitations of conventional controllers, particularly in handling nonlinear stochastic control problems [5, 6]. DRL enables the training of real-time, model-free, self-learning and adaptive controllers, making it highly suitable for complex process industries. In particular, DRL-based controllers can avoid the need for hand-crafted feature descriptors, controller tuning, deriving control laws and developing complex mathematical models.

One challenge with using DRL for process control is that most previous research on the topic has focused on replacing the conventional controller, hence, relying entirely on DRL [5, 7]. This strategy poses a risk to safety-critical systems because:

This is an open access article under the terms of the [Creative Commons Attribution](https://creativecommons.org/licenses/by/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2026 The Author(s). *IET Control Theory & Applications* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

(i) DRL focuses on achieving the goal without considering stability, which could lead to catastrophic scenarios, (ii) Vanilla DRL implementations often assume full observability, though specialized variants exist for partial observability, which is rarely the case, and (iii) DRL is a black-box approach, making it difficult to predict behaviour in abnormal situations. As an initial basis for our work, we adopt a hybrid architecture that combines DRL with a conventional controller, known as residual policy learning (RPL) [8], to develop a hybrid control strategy [6, 9]. The expert policy acts as a safety net and a correction factor, particularly in cases of unobserved states where the DRL agent would fail to optimize, which is demonstrated in our experimental results. The resulting synchronous policy is the superposition of both control strategies as presented in [10, 11].

A second challenge in applying DRL directly to safety-critical applications is that random exploration is both expensive to collect and prone to catastrophic failures. To mitigate this, we extend the RPL approach by incorporating the cycle of learning (CoL) methodology [7]. CoL improves behavioural cloning (BC), where a base policy is trained to imitate the expert policy (conventional controller or human expert) before DRL exploration. Within our framework, CoL reduces the reliance on unsafe random exploration and limited data, allowing the agent to learn efficiently while minimizing the risk of critical failures.

A third challenge in applying DRL to complex and continuous state-action spaces—such as, real-world processes with multiple input variables—is that it learns from the entire dataset rather than focusing on the most relevant data, making convergence more difficult. Using an RPL framework, one way to reduce the challenge of learning in a massive state space is to split the state space into parts that the conventional controller can manage and parts where it struggles (abnormal situations), using the specialized reinforcement learning agent (SRLA) proposed by the authors in [12]. In this work, we adopt the SRLA approach and further extend it to continuous state-action spaces and in the context of CoL with the RPL framework. Furthermore, through our proposed framework for SRLA, we employ an input–output hidden Markov model (IOHMM) to autonomously identify abnormal or ‘low-value’ states, allowing the DRL agent to focus on learning the optimal policy in those targeted states. This approach is analogous to the options methodology [13], where each option identifies a specialization criterion within the hierarchical structure of reinforcement learning. The IOHMM approach is specialized for scenarios where identifying and handling abnormal states is essential. In contrast, the options methodology is a more general framework for hierarchical reinforcement learning (HRL), which can be applied to a wide range of tasks but may not focus explicitly on abnormal state detection and may not be as specialized for rare events.

This paper performs an investigation of the utility of specialized RL in industrial settings with pre-existing control systems. The main contributions of this paper are:

1. A novel integration of CoL, RPL and SRLA, creating an efficient hybrid control framework that addresses challenges inherent in both conventional and DRL approaches. Our method provides a corrective term (via RPL) while enabling safe DRL exploration through CoL.
2. Generalization of SRLA to handle continuous action spaces.
3. Introduction of a state-value function for target state identification to autonomously identify abnormal or ‘low-value’ states for reinforcement learning activation and specialization.

While our method builds on previous works [8, 12], the novelty lies in: (i) integrating RPL with CoL for safer and sample-efficient training, and (ii) extending the SRLA with IOHMM to autonomously detect abnormal states in continuous action spaces. To our knowledge, this combination (CoL + RPL + SRLA) has not been proposed before. We validate this framework on the Tennessee Eastman process (TEP) control problem [14, 15]. In the first set of experiments (Section 5), the combination of RPL (DRL + conventional controller) with CoL is shown to demonstrate the autonomous synchronization between the controllers. In the second experiment (Section 6), we demonstrate the autonomous activation of the controllers through SRLA, and in the last experiment (Section 7) the performance of the overall proposed hierarchical framework (RPL + CoL + SRLA) is validated through an ablation study in a complex scenario.

In the remainder of this paper, Section 2 discusses prior work and the main contributions of this paper, Section 3 presents the methodological framework, Section 4 discusses the design of experiments, Section 5 presents the results of autonomous synchronization of controllers, Section 6 discusses the results of autonomous activation of the DRL agent, and Section 7 evaluates the comparison of our approach to the state-of-the-art, baseline and traditional controller.

Notation.

- For any finite set A , $|A|$ denotes its cardinality.

2 | Related Work

Several works [5, 16–18] have compared conventional controllers with data-driven and adaptive DRL controllers or decision support systems, noting that while DRL can solve non-linear, complex and stochastic control problems, it has limitations such as a lack of optimality guarantees and data requirements. Furthermore, in process or safety-critical industries, it is not recommended to completely replace the conventional controller and rely on such a learned policy that could fail in cases of unobserved states. Instead, DRL and conventional controllers should be used in complementary ways as suggested by [6]. One proposed approach is a hybrid architecture where the DRL agent handles high-level, offline decision-making while the conventional controller handles low-level, online control decisions, hence working independently. However, our proposed approach in this context involves both controllers working together synchronously.

Safety is a critical concern in DRL, particularly in high-risk domains. Safe DRL approaches, such as those proposed by [19–22], aim to incorporate constraints and risk-aware policies into RL training. These works have explored techniques such as constrained optimization, shielding mechanisms and safety functions to ensure policy safety during training and execution. Our

framework builds upon these safety-driven DRL methodologies by ensuring that the conventional controller acts as an adaptive safeguard that prevents unsafe actions, thus addressing concerns regarding reliability in unobserved conditions.

RPL is a technique proposed by [8] that involves learning a residual on top of an existing controller, which can be more efficient than learning a policy from scratch. It was originally used in the context of robotics with similar work done by [10, 23]. Previous work has used RPL in control problems [9, 24], but it requires retuning the algorithm when the system encounters a disturbance, setpoint change or unobserved states. Our approach proposes and demonstrates a novel RPL configuration, where the conventional controller acts as a safety net and correction factor on top of DRL in unobserved abnormal conditions. Furthermore, the prior work has used a fixed weight ratio for the hybrid mix of superposed policies. In contrast, in our framework, the proportion of the hybrid mix is autonomously determined by the multi-controller system. This hybrid mix differs from the conventional notion of a weighting ratio in the superposition equation. Rather, it involves each agent (controller) autonomously assessing the required input magnitude based on the specific scenario at hand and observing the control input by the other agent.

To enhance DRL performance, [7] proposes integrating BC with DRL—a strategy they refer to as the CoL—which enables efficient policy learning even when data is scarce. This approach requires fewer interactions with the safety-critical systems by initializing the DRL agent with a base policy and then improving the policy in an online setting. Furthermore, in our prior work, presented in [12], we introduced SRLA hierarchical framework to enhance the convergence of DRL in intractable problems by focusing the learning process on targeted state-action spaces, thereby addressing the challenges associated with learning in massive state spaces. By specializing in specific state-action regions, SRLA enables more efficient learning and convergence, particularly in complex and continuous environments such as those encountered in safety-critical control problems. In this work, we integrate both strategies—CoL and learning in specialized state spaces—to enhance the performance of RPL. In addition, in this work, we generalize SRLA, extend its application to a continuous action environment, and propose a method for autonomously identifying the target state through the state value function.

HRL has been explored as a way to enhance the performance of RL in complex tasks by structuring policies at different levels of abstraction. Works such as [25–27] have demonstrated that HRL can improve sample efficiency and learning stability by decomposing tasks into sub-tasks managed by high-level and low-level controllers. In particular, [25] proposes HIRO, a method that allows the higher-level policy to generate subgoals for the lower-level policy, which has been beneficial in robotic control and other domains. Building on HRL concepts, our approach further enhances interpretability and hierarchical states identification by integrating an IOHMM-based framework with a hybrid control architecture that systematically explores control strategies around a baseline policy.

To validate the proposed framework (RPL + CoL + SRLA), we use TEP [14, 15]. In the field of controlling the TEP, previous studies

(such as [28, 29]) have explored the application of DRL-based control methods. However, a prominent challenge identified by these authors was the difficulty in attaining performance levels comparable to traditional controllers. Considering that hybrid control schemes are designed to mitigate the effects of disturbances, they are often associated with adaptive and robust control methods [30]. Such controllers typically depend on accurate first-principles models or well-identified process dynamics to ensure stability and robustness. In contrast, the proposed framework adopts a model-free approach that does not require prior knowledge of the underlying plant dynamics. This distinction is particularly relevant for complex industrial settings with pre-existing control systems, where the dynamics are high-dimensional, non-linear, and partially observable, making analytical modelling either impractical or prohibitively expensive [31]. We conducted a comparative analysis and ablation study to demonstrate the effectiveness of our approach. Our findings demonstrate that our approach surpasses state-of-the-art methodologies in performance and effectiveness. Table 1 summarizes how our work extends prior methods versus the proposed framework.

3 | Methodological Framework

3.1 | Preliminaries

The framework is composed of the following modules: (i) deep reinforcement learning, (ii) residual policy learning, (iii) cycle learning and (iv) an input–output hidden Markov model.

3.1.1 | Deep Reinforcement Learning

DRL offers a robust approach for learning optimal control policies through interactions with an environment modelled as a Markov decision process (MDP), defined by the tuple (S, A, P, R, γ) where:

- S is the set of states representing the possible situations the agent can be in.
- A is the set of actions the agent can take.
- P is the transition probability function $P(s_{t+1}|s_t, a_t)$, defining the likelihood of transitioning to the state s_{t+1} given the current state s_t and action a_t .
- R is the reward function $R(s_t, a_t, s_{t+1})$, indicating the immediate reward received after taking an action in a particular state.
- $\gamma \in (0, 1]$ is the discount factor determining the importance of future rewards.

Upon selecting an action a_t , the environment transitions to a new state s_{t+1} based on the transition probabilities $P(s_{t+1}|s_t, a_t)$, which denote the probability of transitioning to state s_{t+1} given that the current state is s_t and action a_t is taken. In practice, the next state s_{t+1} is generated based on the system dynamics, which may be represented by either a simulator or an analytical system model. These dynamics capture how the environment responds to actions and determine the realization of the transition function.

TABLE 1 | Qualitative comparison of CoL-SDRPRL with state-of-the-art methods. The proposed framework supports guided exploration, hybrid control architecture, specialized policy learning and enhanced sample efficiency.

Method	Continuous state-action	Guided exploration	Hybrid control architecture	Specialized policy learning	Sample Efficiency	Partial observability handling
Vanilla DRL (TD3) [32]	•					
Safe-DDPG [22]	•	•				
HIRO [25]	•			•	•	•
CoL [7]	•	•			•	
RPL [8]	•	•	•		•	
SRLA [12]				•		•
BC-SRLA [33]		•		•	•	•
CoL-SDRPRL (Ours)	•	•	•	•	•	•

Note*, Supported.

Upon transitioning to the new state s_{t+1} , the agent receives a reward r_t from the environment. The agent's goal is to learn a policy π that maximizes the expected cumulative reward over time. This is typically formalized using the notion of the return G_t , which is defined as the discounted sum of instantaneous rewards from time step t onwards, that is,

$$G_t^\pi(s_t) \doteq \sum_{k=0}^{\infty} \gamma^k r_{t+k} = \sum_{k=0}^{\infty} \gamma^k R(s_{t+k}, \pi(s_{t+k}), s_{t+k+1}), \quad (1)$$

where $r_t \doteq R(s_t, a_t, s_{t+1})$ is the realization of the reward function at time t , and actions $a_{t+k} = \pi(s_{t+k})$, $k = 1, \dots$, are selected according to a strategy $\pi : S \rightarrow A$. In this context, the agent's goal is to learn a policy π that maps states to actions, maximizing the expected cumulative reward over time.

The expected total return under policy π is defined as follows:

$$\bar{G}_t^\pi(s_t) \doteq \mathbb{E}_{\tau \sim \pi} [G_t^\pi(s_t)], \quad (2)$$

where $\mathbb{E}_{\tau \sim \pi[\cdot]}$ denotes the expectation operator induced by the stochastic process on the product σ -field (of infinite sequences of action-state pairs) conditioned on the available information up to time t , denoted by $\tau = \{(a_t, s_t), (a_{t+1}, s_{t+1}), \dots\}$, which is generated by policy π . This expected return is not known, and it is evaluated as an average return over sufficiently large sequences of action-state samples.

Building on our previous work ([12, 33]) that employed double deep Q-networks (DDQN) ([34]) for discrete action spaces, we extend the framework to handle continuous action spaces using the twin delayed deep deterministic policy gradient (TD3) algorithm ([32]). TD3 follows a model-free, actor-critic architecture ([35]), where the actor determines the policy and the critic evaluates it to guide improvement. Enhancing the deep deterministic policy gradient (DDPG) method [2], TD3 improves training stability and performance through three key strategies. Clipped

double-Q learning mitigates overestimation bias by training two Q-functions and selecting the smaller value. We will denote these two values by Q_1 and Q_2 . Delayed policy updates, update the policy network less frequently than the Q-functions, reducing divergence during training. Target policy smoothing adds noise to the target action, smoothing the Q-function and making the policy more robust to small perturbations.

The Q values (Q_1 and Q_2) modelled by the critic network approximate the expected total return, denoted by $\bar{G}_t^\pi$, as defined by Equation 2. At any given time t , let s_t denote the current state of the process, while π symbolizes the agent's action policy: function that maps states (S) to actions (A). The action a_t^A is then drawn from this policy. Specifically, a_t^A constitutes an $|A|$ -dimensional tuple signifying the agent's selected action at time step t within the action space A .

To mitigate the issue of moving targets during training—challenge where the target values used to train the policy shift continuously because the critic is also being updated—the target value $\bar{G}_t^\pi$ is approximated using the critic networks, which are updated periodically to enhance stability, as follows:

$$\begin{aligned} \bar{G}_t^{\pi'}(s_t) &\doteq r_t(s_t, \pi'(s_{t+1} | \theta_{\pi'}), s_{t+1}) \\ &+ \gamma \min_{i=1,2} Q'_i(s_{t+1}, \pi'(s_{t+1} | \theta_{\pi'}) | \theta_{Q'_i}). \end{aligned} \quad (3)$$

where π' and Q' refer to the target actor/critic networks. Here, r_t represents the immediate reward at time step t . The critic loss function $\mathcal{L}_Q$ for optimizing the critic parameters θ_{Q_1} and θ_{Q_2} is defined as follows:

$$\begin{aligned} \mathcal{L}_Q(s_t, \theta_{Q_1}, \theta_{Q_2}) &\doteq \frac{1}{2} \sum_{i=1}^2 (Q_i(s_t, \pi(s_t | \theta_\pi) | \theta_{Q_i}) \\ &- \bar{G}_t^{\pi'}(s_t))^2, \end{aligned} \quad (4)$$

that is, it corresponds to the average quadratic difference of the individual critic losses with the target value, which is used to evaluate the impact on the value when we switch strategies from π' to π . Finally, the actor loss function $\mathcal{L}_A$ for optimizing the actor parameters θ_π is defined as follows:

$$\mathcal{L}_A(s_t, \theta_\pi) \doteq -Q_1(s_t, \pi(s_t | \theta_\pi) | \theta_{Q_1}). \quad (5)$$

3.1.2 | Cycle of Learning

The purpose of the CoL is to address the problem of distributional drift. This distributional mismatch is caused by the difference in state distribution between the policy initialized by pretraining through BC and fine-tuning the policy during the online training ([7]). The CoL approach builds on top of BC. BC, as proposed by [36], is a method where the agent tries to imitate the demonstrations provided by the expert behaviour. The loss function of BC ($\mathcal{L}_{BC}$) is defined by:

$$\mathcal{L}_{BC}(s_t, \theta_\pi) \doteq \frac{1}{2} (a_t^E - a_t^A)^2 = \frac{1}{2} (a_t^E - \pi(s_t | \theta_\pi))^2, \quad (6)$$

where a_t^E is the expert policy, same as in Equation 8, $a_t^A \doteq \pi(s_t | \theta_\pi)$ is the DRL agent's behaviour policy, and θ_π represents the parameters of the network that clones a behaviour policy.

CoL integrates BC, actor, and critic loss from Equations 6, 5, 4, respectively, during the pretraining as well as during the training to minimize the distribution drift. The overall CoL loss function is defined as:

$$\begin{aligned} \mathcal{L}_{CoL}(s_t, \theta_Q, \theta_\pi) &\doteq \mathcal{L}_{BC}(s_t, \theta_\pi) \\ &+ \mathcal{L}_A(s_t, \theta_\pi) + \mathcal{L}_Q(s_t, \theta_Q). \end{aligned} \quad (7)$$

3.1.3 | Residual Policy Learning

RPL, presented in [8], is a methodology that is designed to improve imperfect and non-differentiable policies by learning a residual factor on top of a policy. Under such a setup, a combined action is derived in the form:

$$a_t^C = a_t^E + a_t^A, \quad (8)$$

where a_t^E is the expert's action (i.e., derived from a conventional controller or a human-generated policy), a_t^A is the DRL agent's action generated by policy π , that is, $a_t^A = \pi(s_t | \theta_\pi)$, and a_t^C is the combined action as represented by the superposition of the baseline/expert policy and the agent policy. In the proposed framework, the proportion of this hybrid mix is autonomously determined by a multi-controller system (consisting of the PID controller and the DRL agent), with each entity independently assessing the required input magnitude based on the scenario and observing the control input of the other entity. PID was chosen because of its ubiquity in industrial practice and its availability in the TEP benchmark. However, the framework is not restricted to PID; other controllers, such as MPC, can be incorporated as the expert baseline within the RPL structure.

3.1.4 | Input–Output Hidden Markov Model

We also consider the IOHMM architecture, as introduced by [37]. Such architecture assumes that observed data Y_t follow *patterns* (represented by Gaussian distributions) that are evolving with time under the Markovian property (i.e., the current pattern, at time t , depends on the previous one, at time $t-1$, as well as the control inputs at time t , U_t). Such patterns are usually referred to as *hidden states* and the objective is to determine the most probable hidden state or sequence of hidden states given a sequence of inputs and observations. Hidden states can represent critical operating conditions of the system, which may be necessary for identifying limitations in conventional controllers. Following the Markovian property, the probability that hidden state $s^{(i)} \in S$ applies at time t , briefly denoted (ζ_t), is expressed as follows:

$$\zeta_t(i) \doteq P[x_t = s^{(i)} | x_{t-1}, U_t, Y_t, \lambda], \quad i = 1, \dots, N, \quad (9)$$

where i ranges from 1 to N . The variable t denotes the current time step in the sequence. Here, U_t represents the input vector $U_t \doteq (u_{t,1}, u_{t,2}, \dots, u_{t,m})$ of dimension m , Y_t is the observed vector of variables $Y_t \doteq (y_{t,1}, y_{t,2}, \dots, y_{t,n})$ of dimension n , and λ denotes the trained model parameters, including the initial state, transition, and emission probability matrices. Here m , n represent the dimensionality of the inputs and observations respectively. We will denote the hidden states of the IOHMM model by the enumeration $\hat{S} \doteq \{s^{(1)}, \dots, s^{(N)}\} \subset S$. The probability operator of Equation (9) is calculated on the canonical path space defined by the product σ -field of infinite sequences of inputs and observations of the IOHMM model.

The hidden state at time t , denoted by x_t , is determined by selecting the most probable state at any given time instance based on the sequence of observations from the set of hidden states. In other words,

$$x_t = \underset{1 \leq i \leq N}{\operatorname{argmax}} [\zeta_t(i)]. \quad (10)$$

The IOHMM handles continuous states by discretizing the state space into N hidden states based on the expert trajectory. Transition probabilities between states are estimated using Gaussian mixture models, allowing continuous emission distributions. The implementation details of the IOHMM are provided in [33]. The most probable state index, determined by Equation 10, which is a predictive model to classify hidden states with trainable parameters λ , will determine the desired state of interest to train and activate the DRL agent within the RPL framework.

3.2 | CoL-Based Specialized Deep Residual Policy RL

In this section, we set the framework that synergistically combines the four components described in Section 3.1 and is illustrated in Figure 1. The left part reflects offline training of the methodology, whereas the right part illustrates the online training phase.

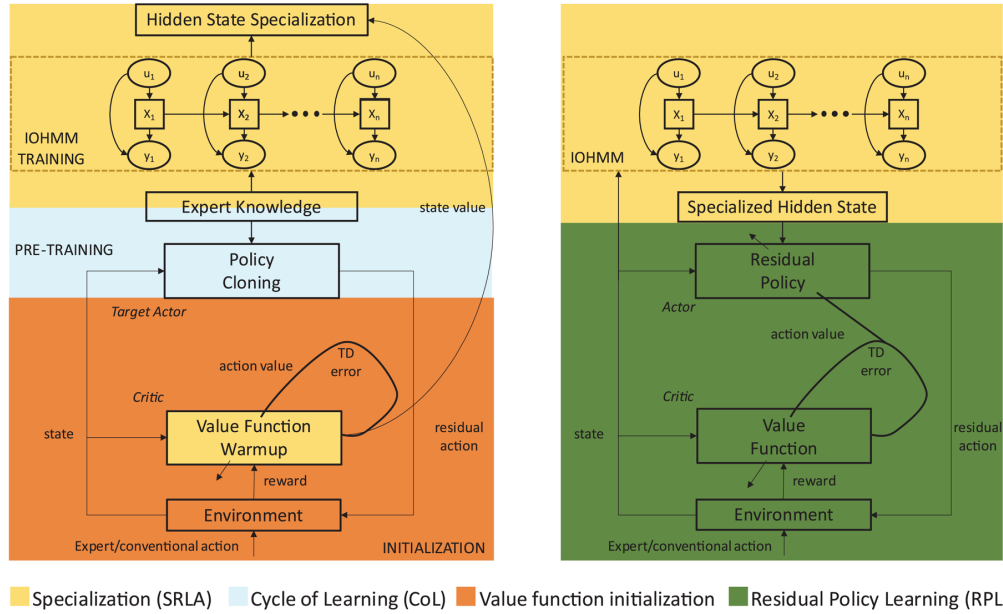


FIGURE 1 | (Left) Offline phase: Policy initialization via expert trajectory, IOHMM training, and state classification. (Right) Online phase: Specialized training and activation of the DRL agent based on identified target states.

In the offline phase, CoL is indicated by the ‘blue divot’ region and the initialization of the value function and the critic is indicated by the ‘red dotted diamond grid’ region. The offline training phase results in a target critic network, providing the output Q_1 , parameterized by θ_{Q_1} , and an actor network that provides the policy $\pi(\cdot|\theta_\pi)$ parameterized by θ_π . In the offline training phase, the policy π is trained to match the operation of the PID controller or expert, that is, $\pi(s_t|\theta_\pi) \approx a_t^E$.

In the online phase of the residual policy learning (RPL), the critic network Q_1 and the actor network π constitute the starting points for the corresponding critic and actor, respectively. The actor and critic network are then periodically re-trained using the updated data (indicated by the ‘green dotted’ region). To simplify notation, we will use Q to denote the updated critic output during the online phase. We will also denote by θ_Q and θ_π the corresponding parameterizations of the critic and actor networks at any given time during the online training phase. The new trained policy provides a (residual) policy $\tilde{\pi}(s_t|\theta_\pi) = a_t^A$, which is combined with the PID controller output to produce the final action, $a_t^C = a_t^E + a_t^A$.

The residual policy is only applied under certain critical conditions. The criticality of an operating condition is defined by the specialization components. The specialization components are indicated with the ‘yellow outlined diamond grid’ regions. During the offline phase, the IOHMM is trained to detect the hidden states (or operating conditions) of the observed process data. During the online phase, the trained IOHMM and value function Q are used to: (a) detect the current operating condition (or hidden state) s^* , and (b) evaluate the critical significance of that state.

The details of the overall setup are discussed in more detail in the forthcoming subsections.

3.2.1 | Offline Policy and IOHMM Training and Initialization

During the offline training phase, the *expert trajectory* serves two purposes: (i) behavioural cloning of the expert policy through CoL and (ii) fitting the IOHMM model. The expert trajectory (B^E) data is collected by simulating the system under both normal and abnormal operating conditions. This involves running simulations and recording the system’s state along with the corresponding actions executed by a conventional controller (e.g., a PID controller). The resulting dataset is formed by aggregating these state-action pairs. The PID controller is not manually optimized for unknown plant dynamics but is based on the standard TEP simulator configuration. This baseline expert trajectory is used for two purposes: (i) behavioural cloning during the CoL phase and (ii) IOHMM training for abnormal state detection. Thus, PID serves as a safety net and baseline rather than the ultimate optimized design.

The next state s_{t+1} is generated based on the underlying system dynamics, which may be represented by either a simulator or a system model. These dynamics determine the environment’s transition behaviour in response to the applied action a_t , providing a realization of the transition probability $P(s_{t+1} | s_t, a_t)$.

With respect to the BC, the offline training phase will result in a target critic network, which provides an approximation of the value function and is indicated by Q_1 , parameterized by θ_{Q_1} .

Furthermore, BC offline training provides an actor network that provides the policy $\pi(\cdot|\theta_\pi)$ parameterized by θ_π . The policy π is trained to match the operation of the PID controller or expert, that is, $\pi(s_t|\theta_\pi) \sim a_t^E$ for each state s_t .

With respect to the IOHMM training, a set of trained parameters λ is derived (by utilizing a standard expectation-maximization algorithm) that allows us to infer the hidden state probabilities, as indicated by Equation (9). The most probable hidden state corresponds to certain statistics (Gaussian distribution) capturing an averaging behaviour of similar observed behaviour in the historical data. The averaging behaviour (statistics) of each hidden state can then be used to classify its criticality, as described in the following subsection.

3.2.2 | Hidden State Classification

During initialization, another essential step is performed by training an additional state-value function ($V : S \rightarrow \mathbb{R}$) defined as follows:

$$V(s) \doteq \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s, \theta_V \right], \quad s \in S \quad (11)$$

which represents the discounted accumulated reward. Here, s corresponds to the hidden state of the IOHMM model represented in the state space of the DRL agent. Furthermore, $S \doteq \{s^{(1)}, \dots, s^{(M)}\}$ is the finite state space of the IOHMM model. The neural network parameters, denoted by θ_V , are updated based on the mean squared error loss defined as follows:

$$\mathcal{L}_V(s_t, s_{t+1}, \theta_V) \doteq \frac{1}{2} ((r_t + \gamma V(s_{t+1} \mid \theta_V)) - V(s_t \mid \theta_V))^2. \quad (12)$$

The trained model V is used to categorize the hidden states of the IOHMM via their performance. Specifically, it is possible to identify states of poor performance or abnormal behaviour, as those states that exhibit lower expectation of the value function V (referred to as 'low-value states'), denoted as:

$$S^* \doteq \left\{ s^{(i)} \in S : \mathbb{E}[V(s^{(i)})] \leq \bar{V} \right\} \subset S, \quad (13)$$

for some baseline performance $\bar{V} \in \mathbb{R}$ which corresponds to the lowest possible 'acceptable' performance. This process is termed 'hidden state specialization'. Here, IOHMM and value function are utilized to create an SRLA.

The value function can be used to classify hidden states because it provides a measure of the expected reward associated with each state. In the CoL-SDRPRL algorithm, the value function is pre-trained on the expert trajectory, which provides a baseline for expected rewards in the system. By comparing the expected reward of each state to the baseline, the value function can identify states that deviate significantly from the expected behaviour, indicating abnormal or anomalous behaviour. Abnormal states are conditions where process performance deviates from nominal or safe operation (low-value states). These are identified using an IOHMM trained on expert trajectories, combined with a

state-value function that classifies 'low-value' hidden states. When such states are detected, the SRLA activates to provide corrective action. While it is true that a value function can help identify states of low value, the IOHMM model offers additional advantages. Not only does it categorize those abnormal states, which may not easily be detected and efficiently computed by a value function alone, but it also extends the approach to cases involving multiple abnormalities.

3.2.3 | Online Policy Training and Evaluation

During online training and inference, the observed state is fed into the IOHMM for hidden state evaluation. As Figure 1 shows, a classification of the hidden states has already been performed, and the currently detected state can be identified as a normal or abnormal state. If the estimated hidden state is identified as an abnormal state, then the deep residual policy reinforcement learning (DRPRL) is activated to be trained and used in the superposed policy. The DRL agent learns to work synchronously with the expert controller (conventional controller or a human agent) during the specified target states (such as abnormal situations in the process industries). If $s^* \in S$ represents a target or abnormal state, then the action of the DRL agent is applied according to Equation (8).

In those target or abnormal cases, the modified loss function used for training CoL-SDRPRL takes on the following form:

$$\mathcal{L}_{\text{CoL-SDRPRL}}(s^*, \theta_Q, \theta_\pi) = \mathcal{L}_{\text{BC}}(s^*, \theta_\pi) + \mathcal{L}_A(s^*, \theta_\pi) + \mathcal{L}_Q(s^*, \theta_Q). \quad (14)$$

where

$$\begin{aligned} \mathcal{L}_{\text{BC}}(s^*, \theta_\pi) &= \frac{1}{2} (a_t^E - \pi(s^* \mid \theta_\pi))^2 \\ \mathcal{L}_A(s^*, \theta_\pi) &= -Q(s^*, (a_t^E + \tilde{\pi}(s^* \mid \theta_\pi)) \mid \theta_Q) \\ \mathcal{L}_Q(s^*, \theta_Q) &= \frac{1}{2} (Q(s^*, (a_t^E + \tilde{\pi}(s^* \mid \theta_\pi)) \mid \theta_Q) - \tilde{G}_t^{\pi'})^2 \\ \tilde{G}_t^{\pi'}(s^*) &= r_t + \gamma \cdot Q'(s_{t+1}, \tilde{\pi}'(s_{t+1} \mid \theta_{\pi'}) \mid \theta_{Q'}) \end{aligned}$$

where $\tilde{\pi}'$ and Q' denote the target actor and critic networks, respectively. s^* refers to the target state (e.g., abnormal state) identified through IOHMM.

It is important to note that during inference, the baseline (PID) controller is always running. The role of the IOHMM and the DRL agent is to first detect whether a state is abnormal, and only in those cases, it takes over by providing the residual action $a_t^A = \tilde{\pi}(s^* \mid \theta_\pi)$. The finally implemented action is the combined one, indicated by the sum $a_t^E + a_t^A$. In all other cases, only the baseline controller that provides action a_t^E applies (e.g., the PID controller). Furthermore, note that the BC cost forces the DRL agent to learn policies that do not significantly deviate from the baseline controller. This provides a level of security in the selection of actions. Furthermore, this design has the additional advantage that the DRL agent can completely replace the baseline controller if needed. The overall control architecture, including the interaction between the expert (PID) controller, specialized DRL agent and IOHMM-based state filtering, is illustrated in the block diagram in Figure 2.

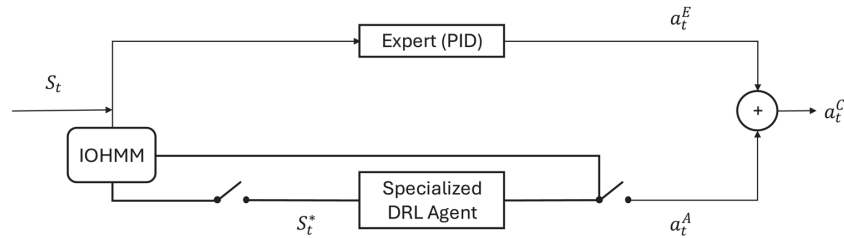


FIGURE 2 | Control system block diagram illustrating the combined control action as a parallel composition of an expert controller and a specialized deep reinforcement learning agent. An input-output hidden Markov model (IOHMM) acts as a state filter, enabling the DRL agent only when relevant system states are detected, thereby allowing its contribution to the overall control signal.

ALGORITHM 1 | CoL-SDRPRL Pseudocode

Offline Training and Initialization

Input: B^E : Expert trajectory buffer
 N : Number of hidden states

- 1: **for** pre-training steps = 1, ..., T **do**
- 2: Compute $\mathcal{L}_{\text{CoL}}$ using Equation (7) and optimize
- 3: **end for**
- 4: **while** expectation-maximization tolerance $> 1 \times 10^{-1}$ **do**
- 5: Compute expectation-maximization ζ (Equation (9))
- 6: **end while**
- 7: **for** warmup steps = 1, ..., W **do**
- 8: Compute L_V using Equation (12) and optimize
- 9: **end for**

Output: $\hat{s}$: Hidden states classification through V ,
 s^* : Specialized hidden state

Online Training and Inference

Input: B^E : Expert trajectory buffer,
 B^A : DRL buffer,
Pre-trained Q and π

- 10: **for** training steps = 1, ..., L **do**
- 11: Compute x_t using Equation (10).
- 12: **if** s_t belongs to s^* as predicted by x_t and categorized by V **then**
- 13: $a_t^C = a_t^E + a_t^A$ (Equation (8))
- 14: Batch = $0.25 \times B^E + 0.75 \times B^A$
- 15: Minimize $\mathcal{L}_{\text{CoL-SDRPRL}}$ using Equation (14)
- 16: **else**
- 17: Take expert's action (a_t^E)
- 18: **end if**
- 19: **end for**

Output: $\hat{\pi}(s^* | \theta_{\pi})$: Residual policy function for the filtered states,
 a_t^C : Hybrid control action

Switching is governed by the IOHMM hidden state classification and the state-value function. In normal operating states, only the PID baseline is active. When an abnormal state is detected, the DRL agent is activated and provides residual corrective action. Abnormal states are defined as those associated with large costs in the value function, which indirectly indicate the inadequacy of the PID controller in maintaining desired performance. Consequently, the RL agent intervenes only in states where the PID control signals are insufficient, ensuring that learning and control efforts are focused on critical regions of the state space. The algorithm of the proposed framework is shown as pseudocode in1, where x_t is the hidden state representation and s_t is the corresponding state for the DRL agent. The source code is available at the [project repository](https://github.com/your-repo)¹.

3.2.4 | Performance Guarantees of CoL-SDRPRL

The performance of the CoL-SDRPRL strategy will depend on the accuracy of the trained actor and critic networks. The CoL-SDRPRL agent performance will not depend on the detection accuracy of the IOHMM model, since it will apply and train over any given state that is considered a target or abnormal state. Its performance is dependent on its ability to adapt to the expert's behaviour (critic network) and accurately track the impact of the selected residual actions (value network).

Given that the hidden states of the IOHMM model correspond to Gaussian distributions, the set of DRL states that are assigned to a target/abnormal hidden state will correspond to a closed subset of the state space S (due to the continuity of the Gaussian distributions). Let $S^* \subset S$ correspond to this closed subset that has been assigned to the target hidden state by the IOHMM model. The DRL agent applies and assigns the residual policy only when the detected state satisfies $s \in S^*$, while in all other cases the expert's controller applies directly. This specialization in the application of the CoL-SDRPRL serves two primary purposes: (a) it provides a level of security, since the expert's controller applies to all cases (i.e., for all states in $S \setminus S^*$); (b) it allows for faster training of the actor-critic networks since both training and inference are restricted to the states in S^* .

We should expect that the performance of the CoL-SDRPRL strategy will only differ within the states S^* , while in all other cases the performance will be identical to the expert's controller. The performance of the DRL agent within the subset S^* can then follow the existing convergence analysis results of standard actor-critic algorithms, such as the local stability analysis presented in [38]. Presenting such analysis here goes outside the scope of this paper, whose contribution lies primarily in the design of the CoL-SDRPRL strategy.

The provable convergence guarantees of the proposed setup go beyond the scope of this paper, where the intention is to primarily investigate the utility of RL in an industrial environment. (Convergence guarantees of actor-critic methods (e.g., soft actor-critic, TD3) provide theoretical support in finite action spaces [39]. However, convergence in continuous domains remains an open research challenge.) Even though convergence guarantees cannot be explicitly claimed, the proposed design has incorporated a combination of safety features that could define a scientific

direction in addressing this issue in industrial environments. Specifically,

1. Constraining RL exploration via CoL.
2. The proposed approach also contributes to the safety of implementing the RL agent, by specializing to different operating conditions (the states of the IOHMM models), which also defines a direction in ensuring convergence guarantees of the model-free actor-critic methods in continuous action spaces (by restricting the RL activation through the IOHMM specialization).
3. Maintaining PID as a safeguard.

The DRL component is activated exclusively during abnormal or low-value operating conditions, as identified by the IOHMM-based specialization module. During these conditions, the PID controller output is gradually reduced to zero while the DRL agent takes over the control action to perform corrective regulation. Conversely, under normal operating conditions, only the PID controller remains active, and the DRL output remains null. Importantly, if the DRL agent is unable to stabilize or fully correct the system response, the PID controller automatically re-engages, providing a residual corrective action. This bidirectional adaptive switching ensures that at all times, at least one stable controller governs the process, thereby preserving closed-loop stability and guaranteeing safe operation across both normal and abnormal regimes.

3.2.5 | Discussion on the Properties of CoL-SDRPRL

CoL-SDRPRL offers several key features beneficial for real-world, safety-critical, and time-sensitive control applications:

- Safe exploration via expert imitation: Pre-trains DRL on expert behaviour, minimizing unsafe exploration.
- State-space partitioning and sample-efficient learning: IOHMM + value functions identify low-performance states and targets DRL intervention on critical states, optimizing training and system integration.
- Interpretable decision-making: Utilizes IOHMM and the value function to classify abnormal states, offering interpretability and improving system trustworthiness.
- Hybrid control and real-time adaptation: A hybrid control strategy, combining an expert controller with a DRL agent, leverages the stability and reliability of conventional control while optimizing performance through adaptable DRL. It dynamically adjusts weights of the hybrid control architecture to enable synergistic collaboration.

4 | Design of Experiments

In this section, we describe the experimental setup, the simulation environment used, and case studies.

4.1 | Experimental Setup: DRPRL Controller

DRL has several components, and its formulation varies between different environments and requires careful consideration, which we define in this section for our case study in process control.

4.1.1 | State

In a partially observable Markov decision process (POMDP), the observed state alone may be insufficient due to the system's inherent partial observability [5]. Consequently, the state perceived by the DRL agent differs from the actual environment state. A tuple comprising the history of expert actions concatenated with the history of process variables, with a maximum length of l , is used to mitigate this limitation, as shown in Equation 15. The selected history includes the current state (t) and the previous trajectory ($t - 1$), following the approach in [8], where l , the history length, is 0 in our case.

$$s_t := \langle (y_{t-l}, a_{t-l-1}^E), \dots, (y_t, a_{t-1}^E) \rangle \quad (15)$$

4.1.2 | Reward

The objective of an agent in a disturbance rejection problem is to find an optimal policy $\pi_{\theta}^*(s_t)$ that reduces tracking error and stabilizes the process with minimal deviation from the optimal setpoint. This is achieved by integrating the goal into the DRL agent through a reward function r or cost function $-r$, such as the negative l_1 -norm of the setpoint error ([5]). Mathematically, for a system with m_y process variables as inputs, it is shown in Equation 16. The hybrid signal integrates PID's stable baseline response with DRL's fast disturbance rejection. The reward (cost) function is defined as the negative L1-norm of setpoint tracking error, directly penalizing deviations and incentivizing disturbance elimination. This ensures both controllers complement each other in mitigating disturbances.

$$r_t(s_t, a_t^A, s_{t+1}) = - \sum_{i=1}^{m_y} |y_{i,t} - y_{i,sp}| \quad (16)$$

4.1.3 | Layer Architecture

The actor in the system uses two long short-term memory [40] layers with 64 and 32 neurons, respectively, to incorporate POMDP state history and a fully connected neural network layer with a single sigmoid output unit. The critic is composed of three fully connected layers with 64 ReLUs, 32 ReLUs and a single linear unit in the output layer.

4.2 | Experimental Environment: Tennessee Eastman Process

We use the revised Simulink version of the TEP plant [15] for validation and modify it for our case studies. The TEP is a widely used industrial process benchmark problem for chemical

process control and optimization. It simulates a complex chemical plant with multiple interconnected units. The process presents challenges such as non-linear dynamics, constraints and disturbances, making it ideal for testing control strategies, fault detection and machine learning applications in industrial automation. The process has 41 measurements (process variables) and 12 controllers (manipulated variables) (see Tables 3–5 in [14]).

4.2.1 | Single-Input Single-Output Setup (SISO)

In this setup, the DRPRL controller takes a single process variable, ‘feed flow A’, as the input state and is affected by a step disturbance, ‘feed loss A’. A single manipulated variable, ‘valve position feed A’, is replaced by our proposed hybrid controller responsible for controlling the process variable and minimizing the disturbance. It further demonstrates the possibility of integrating our framework within different controllers.

4.2.2 | Multiple-Input Single-Output Setup (MISO)

In this setup, we take all the process variables (41) as the input state to the DRPRL controller. The disturbance and the manipulated variable replaced by the framework remain the same as in the previous setup.

4.3 | Case Studies

In the following case studies, the simulation runs with a step size of 0.01 simulation hours, during which the DRL agent observes the policy’s state information and reward. The control strategy is evaluated after every five episodes and after training at every step in between. The experiments are divided into three sets to address different research questions: (i) how the hybrid structure (RPL + CoL) works and complements each other, (ii) validating the convergence of an intractable continuous control problem using a specialized DRL agent, and (iii) comparing the proposed controller with baselines and examining the sensitivity of each element (RPL, CoL, and SRLA). The hyperparameters used in the experiments are available in Appendix A.

5 | Experiment 1: Autonomous Synchronization of Controllers

This set of experiments uses a SISO setup with the RPL + CoL framework without SRLA. The simulation runtime is 5 h, and the disturbance’s start and end times are randomized between the first and the second half of the simulation. Further, to verify the robustness and the complementary effects of the hybrid architecture, the disturbance magnitude is varied from 65% (over which the DRL² and DRPRL³ agent is trained on) to 70% (unobserved state for the DRL and DRPRL agent). These specific percentages were chosen because these were the limits for the plant after which it observes an emergency shutdown as the controller limits are reached.

In the TEP environment, the term ‘Control Valve Opening’ refers to the manipulated variable, representing the valve opening percentage that controls the flow of reactant A. This variable directly reflects the control signal applied by the hybrid controller and is used in the figures to illustrate the system’s response and controller behaviour under varying disturbance conditions. The PID controller, DRL controller, and DRPRL controller responses are plotted in Figures 3a,b, 4a,b, and 5a,b to disturbances of 65% and 70% each. The reference value or the setpoint for the feed flow is 0.27, and the start and end of the step disturbances can be visualized from the two peaks in the figures. The output of each controller (PID and DRL), extracted from the superposed signal of the DRPRL agent at each time step, and their synchronization effects are demonstrated in Figure 6a,b; for 65% and 70% disturbance, respectively.

5.1 | Results and Discussion

For the first set of experiments, the main objectives were twofold: (i) to evaluate the performance of the DRL controller when operating independently, particularly under unobserved disturbance conditions, and (ii) to examine the synchronization dynamics between the DRL and the conventional controller within the proposed hybrid architecture, as well as its robustness in unobserved scenarios. The PID controller exhibited a delayed response in disturbance rejection, which, in the case of a 70% disturbance (Figure 3b), posed a risk of triggering an emergency shutdown due to its limited corrective capacity. In contrast, the DRL controller achieved rapid disturbance rejection and system stabilization within a few time steps for the 65% disturbance case (Figure 4a). However, when subjected to a 70% disturbance (Figure 4b), the DRL controller is not able to reach the desired feed-flow and fails to fully restore the system to its nominal operating point once the disturbance magnitude exceeds the trained range. The system remains slightly offset from the desired setpoint. This demonstrates that while DRL achieves excellent tracking in known regimes, it lacks robustness in unobserved rare operating conditions.

Within the hybrid architecture, an initialization phase⁴ can be observed during which the DRPRL controller achieves synchronization between the DRL and PID components, as shown in Figure 5a. After synchronization, the DRL controller takes over the PID, and the PID response reaches zero, as shown in Figure 6a. The benefits of this autonomous synchronization mechanism are further highlighted in Figure 5a, where, under an increased disturbance magnitude, the PID controller automatically re-engages to assist the DRL agent. This cooperative behaviour provides the additional corrective effort required to compensate for residual errors, thereby ensuring system stability and maintaining smooth process operation, as shown in Figure 6b. The DRL component ensures fast dynamic response, while the PID component contributes integral stability and safe recovery under unobserved conditions. This synergy allows the system to retain both adaptability and robustness.

To compare the results of the two baselines (PID and DRL controller alone) with the DRPRL controller, the simulation runs were evaluated for both cases of disturbances, and the average of episode returns was calculated for 10 runs after 150 episodes of

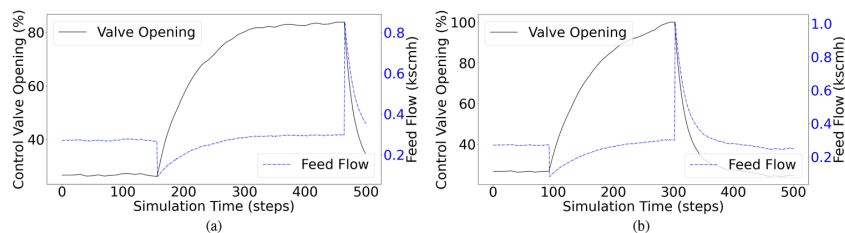


FIGURE 3 | PID controller response to step disturbances. (a) 65% disturbance: The PID controller exhibits a delayed reaction time, gradually stabilizing the process variable. (b) 70% disturbance: The delayed response risks an emergency shutdown as the controller reaches operational limits.

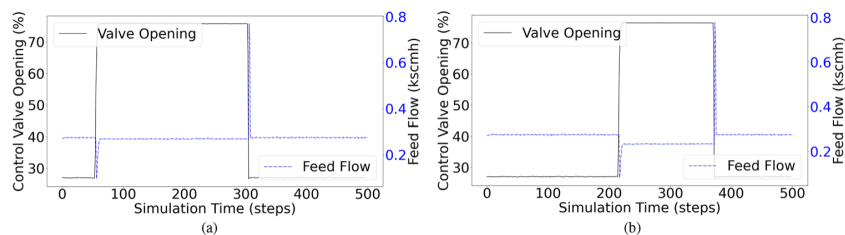


FIGURE 4 | DRL controller response to disturbances. (a) 65% disturbance: The DRL controller achieves rapid disturbance rejection, stabilizing the system efficiently. (b) 70% disturbance: The DRL controller struggles to stabilize the system under unobserved disturbances, demonstrating the need for residual correction.

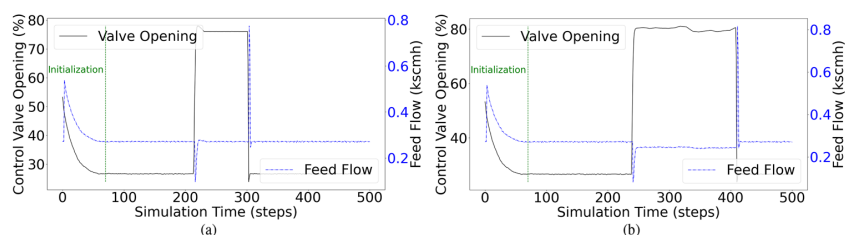


FIGURE 5 | Hybrid DRPRL controller response to disturbances. (a) 65% disturbance: The hybrid approach successfully synchronizes both controllers, allowing transition between DRL and PID responses. (b) 70% disturbance: The PID controller reactivates to assist DRL under unobserved system state, preventing system failure and ensuring process stability.

training steps. The evaluation was based on the episode returns after the initialization phase of the DRPRL case, and the same was applied to the other cases having the same step-size delay for the accumulation of returns. The results are shown in Table 2, where the DRPRL architecture outperforms the baseline controllers in a SISO process control setup.

6 | Experiment 2: Autonomous Activation of CoL-SDRPRL Agent

In this set of experiments, the MISO setup is utilized. The simulation runs for 2.5 h in the simulation runtime and the disturbance starts randomly during the first half of the simulation and lasts till

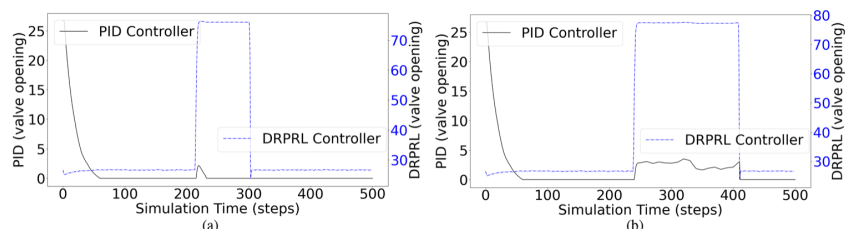


FIGURE 6 | Synchronization of PID and DRL controllers within the DRPRL framework. (a) For 65% disturbance, DRL dominates while PID remains inactive post-synchronization. (b) For 70% disturbance, PID assists DRL in compensating for additional correction factors in unobserved system state.

TABLE 2 | Performance comparison of PID, CoL, and CoL+DRPRL controllers under different disturbance levels. The hybrid CoL+DRPRL controller outperforms both baselines, particularly in handling unobserved disturbances.

Architecture	Average episode reward (10 runs)	
	Disturbance Magnitude	
	65%	70%
PID	-17.71	-23.54
CoL	-2.58	-12.25
CoL+DRPRL	-1.85	-6.25

the end of the simulation. For these experiments, the convergence criteria are observed during normal and abnormal situations.

6.1 | CoL-Based DRPRL Controller

The experiment runs for 250 episodes in this setup without having the specialization phase, and the CoL-DRPRL is trained on the entire dataset. The controller responds to the normal and abnormal states and is unable to optimize in both situations.

6.1.1 | CoL-Based Specialized DRPRL Controller

With the proposed framework (CoL-SDRPRL), the IOHMM is trained on the expert data (time series of process variables). The distribution of the hidden states predicted by IOHMM (using Equation 10) is plotted on the process variable of 'feed flow A' as shown in Figure B1 of Appendix B. It was observed that the IOHMM identified the state that was associated with the abnormal situations (state 3), however, manual identification of such state is not practical in realistic scenarios, where the disturbance criteria are not known in advance and with a multivariate process it becomes difficult to isolate the variables that inherit disturbance. Therefore, the interpretation through the state value functions (V) is used for classifying the abnormal state automatically, as defined in our framework. The abnormal situation is evaluated by the mean state value function with the negative score as shown in Table C1 of Appendix C. State 3 was identified as the state corresponding to the abnormal situations, and the DRPRL is specialized on this state and activates once this state is predicted.

6.1.2 | Results and Discussion

For the second set of experiments, the effect of autonomous switching between the DRL agent and the PID controller is observed, and the comparison of the efficient training on the relevant data with the DRL agent trained on the entire data is validated. The response of the CoL-SDRPRL and autonomous activation of the agent during normal and abnormal scenarios is shown in Figure 7 after being trained on the relevant data for 50 episodes. The agent without specialization did not converge to the optimal policy and further deviated and performed poorly from

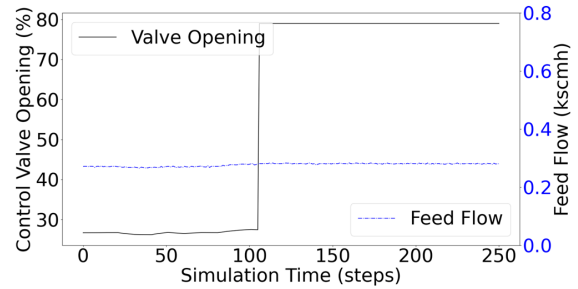


FIGURE 7 | Autonomous activation of the CoL-SDRPRL agent. The DRL controller remains inactive during normal operation, while the PID controller maintains stability. Upon detecting an abnormal event, the CoL-SDRPRL agent is activated preemptively, mitigating the disturbance before system disturbance occurs.

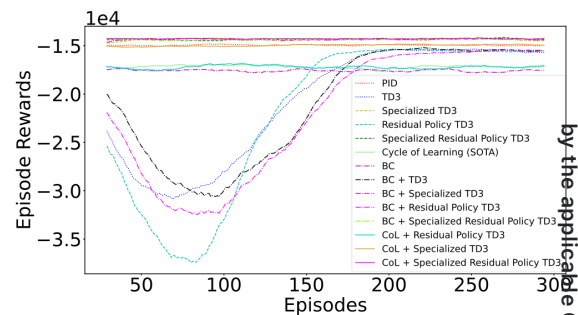


FIGURE 8 | Episode reward analysis for the ablation study. The CoL-SDRPRL framework with specialization exhibits superior convergence and stability compared to other configurations, demonstrating its effectiveness in focusing learning on critical states.

the expert policy. However, in Figure 7, the PID controller works well in the normal state, and the CoL-SDRPRL agent is activated once an abnormal condition is observed. Furthermore, another important phenomenon can be observed as compared to the previous experimental results from Section 5, the CoL-SDRPRL framework identifies the abnormal situation before acting on it and switches to the required controller. This prevents the disturbance from occurring and the plant operates at its normal behaviour at all times, as opposed to observing the sudden spike and then recovering from it.

7 | Experiment 3: Ablation Study

In this ablation study, we use the MISO setup to systematically assess the contribution of each component within our proposed CoL-SDRPRL framework. Specifically, we evaluate different combinations of the methodological elements—BC, CoL, specialization via the extended IOHMM, RPL, and the underlying DRL (TD3) controller—to determine the relevance and impact of each factor. The disturbance level is set at 65%, and all configurations are trained with the same seed for 300 episodes. The initial experimental analysis and training curves are presented in Figure 8. Our results reveal that using DRL (TD3) or an RPL framework without the incorporation of specialization

TABLE 3 | Ablation study results assessing the impact of different framework components. The CoL-SDRPRL configuration achieves the highest average reward, indicating the significance of combining CoL, RPL and specialization.

Method	Pre-training loss	Training loss	Controller	Average reward
CoL+SRPTD3	$\mathcal{L}_{\text{CoL-SDRPRL}}$	$\mathcal{L}_{\text{CoL-SDRPRL}}$	$a^E + a^A$	-14274 ± 350
BC+SRPTD3	$\mathcal{L}_{\text{BC}}$	$\mathcal{L}_Q + \mathcal{L}_A$	$a^E + a^A$	-14340 ± 324
SRPTD3	None	$\mathcal{L}_Q + \mathcal{L}_A$	$a^E + a^A$	-14342 ± 334
STD3	None	$\mathcal{L}_Q + \mathcal{L}_A$	a^A	-14364 ± 348
BC+STD3	$\mathcal{L}_{\text{BC}}$	$\mathcal{L}_Q + \mathcal{L}_A$	a^A	-14370 ± 360
PID	None	None	a^E	-14942 ± 323
CoL+STD3	$\mathcal{L}_{\text{CoL}}$	$\mathcal{L}_{\text{CoL}}$	a^A	-14956 ± 294
CoL	$\mathcal{L}_{\text{CoL}}$	$\mathcal{L}_{\text{CoL}}$	a^A	-17178 ± 701
CoL+RPTD3	$\mathcal{L}_{\text{CoL}}$	$\mathcal{L}_{\text{CoL}}$	$a^E + a^A$	-17188 ± 662
BC	$\mathcal{L}_{\text{BC}}$	None	a^A	-17577 ± 735
TD3	None	$\mathcal{L}_Q + \mathcal{L}_A$	a^A	-20702 ± 6008
BC+TD3	$\mathcal{L}_{\text{BC}}$	$\mathcal{L}_Q + \mathcal{L}_A$	a^A	-20716 ± 5965
BC+RPTD3	$\mathcal{L}_{\text{BC}}$	$\mathcal{L}_Q + \mathcal{L}_A$	$a^E + a^A$	-21745 ± 6634
RPTD3	None	$\mathcal{L}_Q + \mathcal{L}_A$	$a^E + a^A$	-21845 ± 8230

or CoL leads to inefficient training, even when pre-training with BC is employed, the resulting distribution shift hinders performance, which is not acceptable in safety-critical applications. Conversely, configurations that integrate the specialization component consistently outperform other setups. In particular, the best-performing configurations combine policy initialization via CoL pre-training on the expert trajectory with the activation of an SRLA that focuses the learning process on targeted state-action spaces through RPL.

7.1 | Results and Discussion

In our ablation study, we systematically evaluated various configurations of the proposed methodology by isolating or removing individual components. Based on the final evaluation, we observed that when the residual policy component was omitted, the policy initially exhibited poor performance, only converging to its optimal behaviour after several episodes. Table 3 presents a detailed breakdown of each architecture's performance and key features. Configurations marked with an underlined 'S' include the specialization module, and these setups consistently outperformed those without it. Furthermore, among the specialized configurations, those incorporating RPL achieved superior performance, with additional gains when the CoL approach was applied. These results highlight the effectiveness of each component in enhancing overall control performance.

The mechanism by which specialization influences policy optimization involves the use of the IOHMM and state-value function (V_θ) to automatically identify abnormal or low-level states, allowing the RPL agent to focus on learning the optimal policy in those targeted states. This specialization approach helps optimise policy within the region of interest, such as abnormality, where the reinforcement learning agent is required and activated. Furthermore, the specialization components are utilized only in

the online phase to prune out the relevant desired data and guide the reinforcement learning agent to focus on learning the optimal policy in the targeted abnormal states. Although specialization does not directly impact policy initialization, it plays a crucial role in shaping the behaviour of the reinforcement learning agent during the online training phase.

8 | Conclusion and Limitations

This study investigated the performance of RPL enhanced with a CoL and an SRLA, integrating a conventional PID controller with a DRL controller to address safety-critical and intractable continuous control problems. Three sets of experiments were conducted to evaluate the synchronization between controllers under unobserved conditions, analyse the convergence of the specialized DRL agent, and perform an ablation study assessing the contribution of each component (RPL, CoL and SRLA). The results demonstrated that the DRL controller responded rapidly to disturbances while the PID controller provided residual corrections to ensure stability. The specialization mechanism enabled the DRL agent to focus exclusively on abnormal or low-value states, improving convergence and outperforming baseline methods. Although the framework is more computationally demanding than PID alone, the integration of CoL and IOHMM specialization reduces sample complexity and runtime overhead by limiting learning and inference to relevant abnormal conditions. Current validation is simulation-based using the TEP environment, and future work will include hardware-in-the-loop and real-time testing to evaluate the framework's feasibility for industrial deployment. We also acknowledge that the need for a predefined number of hidden states and discretization can limit scalability, and future work will explore dynamic state partitioning, end-to-end learning, and quantitative risk-based validation to reinforce safety and adaptability in complex environments.

Author Contributions

Ammar Abbas: conceptualization, data curation, investigation, methodology, software, validation, visualization, writing – original draft. **Georgios Chasparis:** conceptualization, methodology, supervision, writing – review and editing. **John Kelleher:** conceptualization, methodology, supervision, writing – review and editing.

Acknowledgements

This research was supported by the Collaborative Intelligence for Safety-Critical Systems (CISC) project; the CISC project which has received funding from the European Union's Horizon 2020 Research and Innovation Program under the Marie Skłodowska-Curie grant agreement no. 955901. The work of J. Kelleher is also partly funded by the ADAPT Centre, which is funded under the Science Foundation Ireland (SFI) Research Centres Program (Grant No. 13/RC/2106 P2). The work of G. Chasparis is also partly funded by the 'COGNERGY' project (Grant No. FO999899014), and by BMK, BMAW, and the State of Upper Austria in the frame of the SCCH competence center INTEGRATE (FFG grant no. 892418) part of the FFG COMET Competence Centers for Excellent Technologies Programme.

Funding

This work was supported by the Science Foundation Ireland (SFI) under Grant No. 13/RC/2106_P2 (ADAPT Centre for Digital Content Technology), by the European Commission's Horizon Europe Marie Skłodowska-Curie Actions under Grant Agreement No. 955901, and by the Austrian Research Promotion Agency (FFG) under Grant No. FO999899014.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are openly available in the GitHub repository CoL-SDRPRL at <https://github.com/ammarn-abbas/CoL-SDRPRL>, with the DOI: <https://doi.org/10.5281/zenodo.10641059>.

Endnotes

¹Project repository: <https://github.com/ammarn-abbas/CoL-SDRPRL>

²DRL agent: actions that include only DRL.

³DRPRL agent: superposed DRL agent's action with the expert policy.

⁴This initialization corresponds to the starting conditions defined within the simulation script.

⁵Project repository: <https://github.com/ammarn-abbas/CoL-SDRPRL>

References

1. A. Tulshyan, C. Garvin, and C. Undey, "Machine-Learning for Biopharmaceutical Batch Process Monitoring With Limited Data," *IFAC-PapersOnLine* 51, no. 18 (2018): 126–131.
2. T. P. Lillicrap, J. J. Hunt, A. Pritzel, et al., "Continuous Control With Deep Reinforcement Learning," paper presented at the 4th International Conference on Learning Representations (ICLR), San Juan, Puerto Rico, May 2–4, 2016.
3. J.-M. Lee and J. H. Lee, "Approximate Dynamic Programming Strategies and Their Applicability for Process Control: A Review and Future Directions," *International Journal of Control, Automation, and Systems* 2, no. 3 (2004): 263–278.
4. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction* (MIT Press, 2018).
5. S. Spielberg, A. Tulshyan, N. P. Lawrence, P. D. Loewen, and R. Bhushan Gopaluni, "Toward Self-Driving Processes: A Deep Reinforce-

ment Learning Approach to Control," *AIChE Journal* 65, no. 10 (2019): e16689.

6. J. Shin, T. A. Badgwell, K.-H. Liu, and J. H. Lee, "Reinforcement Learning—Overview of Recent Progress and Implications for Process Control," *Computers & Chemical Engineering* 127 (2019): 282–294.

7. V. G. Goecks, G. M. Gremillion, V. J. Lawhern, J. Valasek, and N. R. Waytowich, "Integrating Behavior Cloning and Reinforcement Learning for Improved Performance in Dense and Sparse Reward Environments," in *International Conference on Autonomous Agents and MultiAgent Systems (IFAAMAS)*, 2020, 465–473.

8. T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, "Residual Policy Learning," preprint, arXiv, December 15, 2018.

9. A. Hynes, E. P. Sapozhnikova, and I. Dusparic, "Optimising PID Control With Residual Policy Reinforcement Learning," in *Irish Conference on Artificial Intelligence and Cognitive Science (CEUR Workshop Proceedings, 2020)*, 277–288.

10. T. Johannink, S. Bahl, A. Nair, et al., "Residual Reinforcement Learning for Robot Control," in *2019 International Conference on Robotics and Automation (ICRA)* (IEEE, 2019), 6023–6029.

11. A. N. Abbas, G. C. Chasparis, and J. D. Kelleher, "Deep Residual Policy Reinforcement Learning as a Corrective Term in Process Control for Alarm Reduction: A Preliminary Report," in *European Safety and Reliability Conference* (Research Publishing, 2022), 3260–3266.

12. A. N. Abbas, G. C. Chasparis, and J. D. Kelleher, "Interpretable Input Output Hidden Markov Model-Based Deep Reinforcement Learning for the Predictive Maintenance of Turbofan Engines," in *International Conference on Big Data Analytics and Knowledge Discovery (Springer, 2022)*, 133–148.

13. R. S. Sutton, D. Precup, and S. Singh, "Between MDPS and Semi-MDPS: A Framework for Temporal Abstraction in Reinforcement Learning," *Artificial Intelligence* 112, no. 1–2 (1999): 181–211.

14. J. J. Downs and E. F. Vogel, "A Plant-Wide Industrial Process Control Problem," *Computers & Chemical Engineering* 17, no. 3 (1993): 245–255.

15. A. Bathelt, N. L. Ricker, and M. Jelali, "Revision of the Tennessee Eastman Process Model," *IFAC-PapersOnLine* 48, no. 8 (2015): 309–314.

16. R. Nian, J. Liu, and B. Huang, "A Review on Reinforcement Learning Introduction and Applications in Industrial Process Control," *Computers & Chemical Engineering* 139 (2020): 106886.

17. Y. Bao, Y. Zhu, and F. Qian, "A Deep Reinforcement Learning Approach to Improve the Learning Performance in Process Control," *Industrial & Engineering Chemistry Research* 60, no. 15 (2021): 5504–5515.

18. J. Deng, S. Sierla, J. Sun, and V. Vyatkin, "Reinforcement Learning for Industrial Process Control: A Case Study in Flatness Control in Steel Industry," *Computers in Industry* 143 (2022): 103748.

19. J. Garcia and F. Fernández, "A Comprehensive Survey on Safe Reinforcement Learning," *Journal of Machine Learning Research* 16, no. 1 (2015): 1437–1480.

20. F. Berkenkamp, M. Turchetta, A. Schoellig, and A. Krause, "Safe Model-Based Reinforcement Learning With Stability Guarantees," in *Advances in Neural Information Processing Systems* (Curran Associates, Inc., 2017), 908–918.

21. T.-H. Pham, G. D. Magistis, and R. Tachibana, "OptLayer-Practical Constrained Optimization for Deep Reinforcement Learning in the Real World," in *2018 IEEE International Conference on Robotics and Automation (ICRA)* (IEEE, 2018), 6236–6243.

22. G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, and Y. Tassa, "Safe Exploration in Continuous Action Spaces," preprint, arXiv, January 26, 2018.

23. M. Alakuijala, G. Dulac-Arnold, J. Mairal, J. Ponce, and C. Schmid, "Residual Reinforcement Learning From Demonstrations," preprint, arXiv, June 15, 2021.

24. Y. T. Liu, E. Price, M. J. Black, and A. Ahmad, "Deep Residual Reinforcement Learning Based Autonomous Blimp Control," in *IEEE/RSJ International Conference on Intelligent Robots and Systems* (IEEE, 2022), 12566–12573.

25. O. Nachum, S. S. Gu, H. Lee, and S. Levine, "Data-Efficient Hierarchical Reinforcement Learning," *Advances in Neural Information Processing Systems* 31 (2018): 3307–3317.

26. A. S. Vezhnevets, S. Osindero, T. Schaul, et al., "Feudal Networks for Hierarchical Reinforcement Learning," paper presented at the 34th International Conference on Machine Learning (ICML), Sydney, Australia, August 6–11, 2017.

27. A. Levy, R. Platt, K. Saenko, and J. Schulman, "Learning Multi-Level Hierarchies With Hindsight," paper presented at the 7th International Conference on Learning Representations (ICLR), New Orleans, LA, May 6–9, 2019.

28. A. Conradie and C. Aldrich, "Plant-Wide Neurocontrol of the Tennessee Eastman Challenge Process Using Evolutionary Reinforcement Learning," paper presented at the 3rd International Conference on Intelligent Processing and Manufacturing of Materials, Cape Town, South Africa, September 2001.

29. N. Kotarsky and E. Bergvall, "Reinforcement Learning in Industrial Applications," (master's thesis, Lund University, 2020).

30. D. Wang, H. He, and D. Liu, "Adaptive Critic Nonlinear Robust Control: A Survey," *IEEE Transactions on Cybernetics* 47, no. 10 (2017): 3429–3451.

31. L. Wang, X. Li, R. Zhang, and F. Gao, "Reinforcement Learning-Based Optimal Fault-Tolerant Tracking Control of Industrial Processes," *Industrial & Engineering Chemistry Research* 62, no. 39 (2023): 16014–16024.

32. S. Fujimoto, H. Hoof, and D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," in *Proceedings of the 35th International Conference on Machine Learning* (PMLR, 2018), 1587–1596.

33. A. N. Abbas, G. C. Chasparis, and J. D. Kelleher, "Hierarchical Framework for Interpretable and Specialized Deep Reinforcement Learning-Based Predictive Maintenance," *Data & Knowledge Engineering* 149 (2024): 102240.

34. H. Van Hasselt, A. Guez, and D. Silver, "Deep Reinforcement Learning With Double Q-Learning," *AAAI Conference on Artificial Intelligence* 30, no. 1 (2016): 2094–2100.

35. V. Konda and J. Tsitsiklis, "Actor-Critic Algorithms," *Advances in Neural Information Processing Systems* 12 (1999): 1008–1014.

36. D. Michie, M. Bain, and J. Hayes-Miches, "Cognitive Models From Subcognitive Skills," *IEE Control Engineering Series* 44 (1990): 71–99.

37. Y. Bengio and P. Frasconi, "An Input Output HMM Architecture," *Advances in Neural Information Processing Systems* 7 (1994): 427–434.

38. M. Holzleitner, L. Gruber, J. Arjona-Medina, J. Brandstetter, and S. Hochreiter, "Convergence Proof for Actor-Critic Methods Applied to PPO and RUDDER," in *Transactions on Large-Scale Data- and Knowledge-Centered Systems XLVIII*, ed. A. Hameurlain and A. M. Tjoa (Springer, 2021), 105–130.

39. T. Haarnoja, A. Zhou, K. Hartikainen, et al., "Soft Actor-Critic Algorithms and Applications," preprint, arXiv, December 13, 2018.

40. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation* 9, no. 8 (1997): 1735–1780.

41. I. Visser and M. Speekenbrink, "depmixS4: An R Package for Hidden Markov Models," *Journal of Statistical Software* 36 (2010): 1–10.

Appendix A: Training Parameters

IOHMM training is conducted using a library based on [41]. The following parameters were used in our experiments:

- IOHMM settings:
 - Hidden states: $N = 4$
 - Process history length: $l = 1$
 - Replay buffer: Size = 10^5 , batch size = 128
- DRL parameters:
 - Discount factor (γ) = 0.99
 - Soft update rate (τ) = 0.005
 - Policy noise = 0.02, noise clip = 0.05
 - Policy update frequency = 2
 - Exploration noise = 0.01
 - Epsilon action probability (random action selection) = 0.2
- Training pipeline:
 - Pretraining steps = 1000
 - Training steps = 10,000
- Cycle of learning (CoL):
 - Learning rate = 3×10^{-3}
- Neural network training:
 - Actor: Learning rate = 3×10^{-3} , weight decay = 0.1
 - Critic: Learning rate = 3×10^{-4} , weight decay = 0.1
 - Value function: Learning rate = 3×10^{-4} , weight decay = 0.1

The source code is available at the [project repository](#)⁵.

Appendix B: Figures

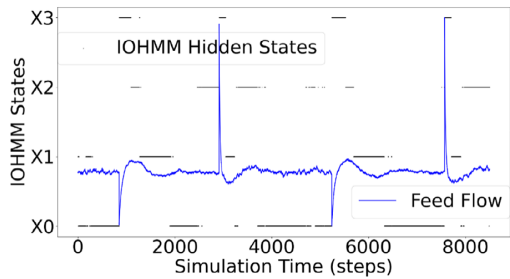


FIGURE B1 | IOHMM state distribution for process variable classification. The trained IOHMM model categorizes hidden states, with the specialized reinforcement learning agent being activated in low-value states, ensuring targeted learning and improved convergence.

Appendix C: Tables

TABLE C1 | Classification of hidden states based on the value function. State X_3 is identified as the critical abnormal state requiring specialized reinforcement learning activation.

IOHMM states	Mean state value	SD (σ)	95% CI high	95% CI low
X_0	7.58	2.30	7.62	7.53
X_1	1.17	7.00	1.43	0.91
X_2	5.39	3.92	5.51	5.27
X_3	−26.08	21.76	−24.86	−27.30